Bibliotech — Database Diagram and Data Dictionary

Project: Bibliotech
Document: Entity-relationship model and table reference
Generated: 2026-10-27
Database: MariaDB / MySQL, InnoDB, `utf8mb4`
Schema sources of truth: `bibliotech/backend/config/init.sql`, `bibliotech/backend/config/migrate_academic_groups.sql`, `bibliotech/backend/config/migrate_refinements.sql`, `bibliotech/backend/config/migrations/001_notifications.sql`

This document describes the schema that the current application code actually uses. It intentionally does not reproduce the older model shown in `Bibliotech.pdf`.

1. Overview

Bibliotech stores users, academic structure, learning materials, and the relationships that drive visibility, review, subscriptions, notifications and auditing.

- **13 tables**
- **18 foreign keys**
- **3 many-to-many tables** (`subject_professors`, `material_subjects`, `material_favorites`) plus unique-combination tables (`subscriptions`, `notifications`)
- Materials can belong to several subjects through `material_subjects`.
- Subjects can have several professors through `subject_professors`.
- Students subscribe to subjects through `subscriptions`; subscribers are the observers notified when new material appears.
- Favorites are stored per user in `material_favorites`; `materials.contador_favoritos` is a denormalized counter and `materials.es_favorito` is a legacy global flag.
- Thematic tags are a separate catalog. They are **not** joined to materials by a foreign key; materials keep a comma-separated `categories` text field.

Table groups

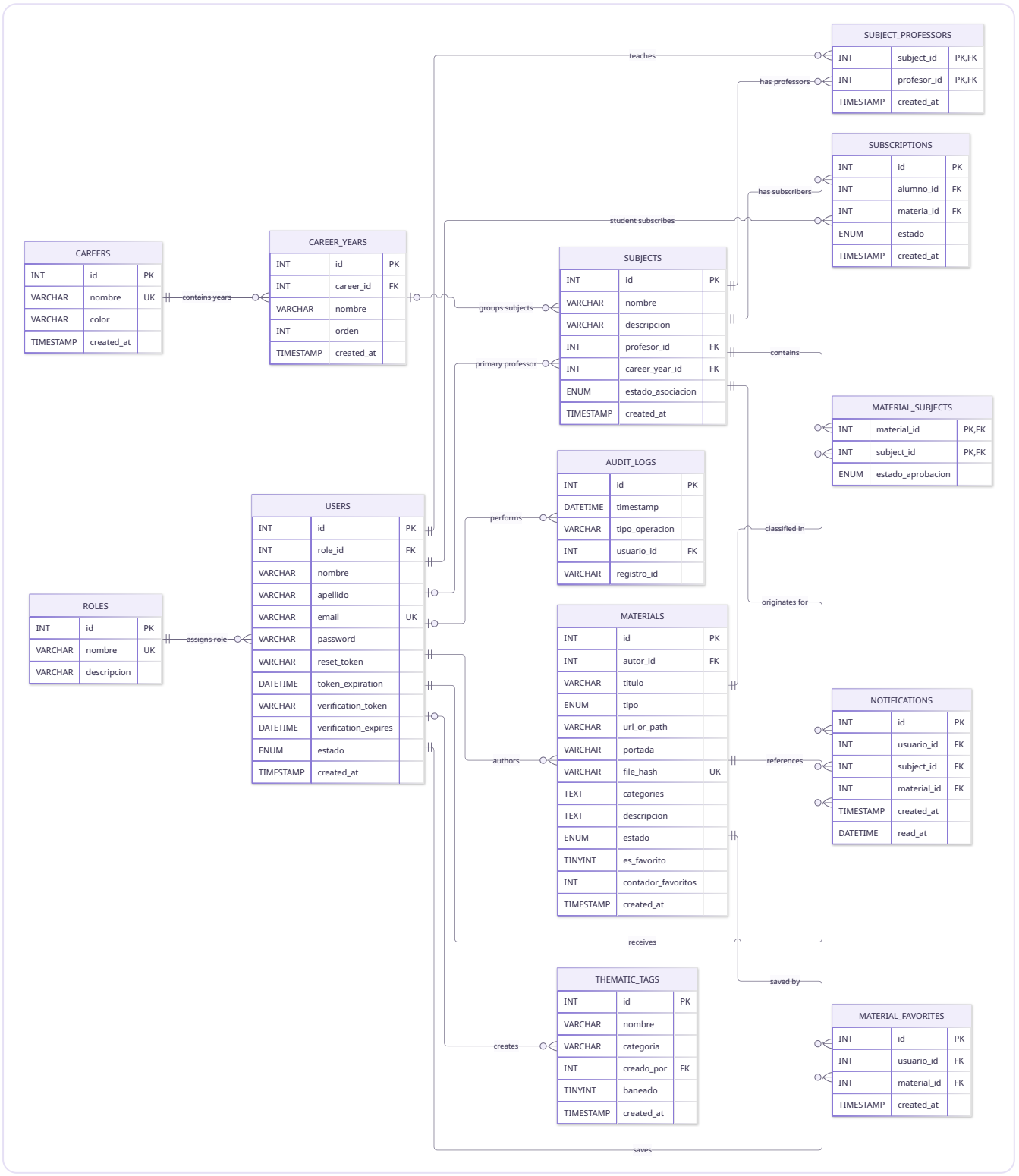
Group	Tables
Identity and access	`roles`, `users`
Academic structure	`careers`, `career_years`, `subjects`, `subject_professors`
Materials	`materials`, `material_subjects`
Student / subject relationship	`subscriptions`
Engagement	`material_favorites`, `thematic_tags`
Notifications and traceability	`notifications`, `audit_logs`

Important modeling decisions

1. **`subjects.profesor_id` is a cached primary professor.** The authoritative many-to-many assignment is `subject_professors`. `subjects.profesor_id` is kept for backwards compatibility and quick display; the repository synchronizes it to the first assigned professor.
2. **`material_favorites` is authoritative for per-user favorites.** `materials.es_favorito` is a legacy/global flag, not the current user's state. `materials.contador_favoritos` is maintained as a cached count.
3. **`audit_logs.registro_id` is polymorphic.** It can point to a user, subject, material, role, career or career year. Because the target table varies, there is deliberately no foreign key on this column.
4. **`materials.file_hash` is unique but nullable.** The SHA-256 hash prevents duplicate uploads while a material is active. When a material is canceled or rejected, file fields are cleared, allowing the same file to be uploaded again later.

5. `thematic_tags` **does not enforce material references.** Tags are a moderated vocabulary. A material's `categories` field is plain text (a comma-separated list), so deleting a tag does not rewrite existing material text.
6. **No database-level role check exists for** `subject_professors`**.** The database guarantees that `profesor_id` references a real user, but the application is responsible for checking that the user has the `Profesor` role.

2. Entity-Relationship Diagram



Reading the cardinalities

- `||--o{` means **exactly one to zero or more**.
- `|o--o{` means **zero or one to zero or more** and is used for nullable foreign keys.
- `PK, FK` indicates a column that is both part of the primary key and a foreign key.

3. Relationship Catalog

#	Parent	Child	Foreign key	Cardinality	On delete	Meaning
1	`roles`	`users`	`users.role_id`	one role → many users	`RESTRICT` / no action	Every user has exactly one role.
2	`careers`	`career_years`	`career_years.career_id`	one career → many years	`CASCADE`	Deleting a career deletes its year levels.
3	`career_years`	`subjects`	`subjects.career_year_id`	zero or one year → many subjects	`SET NULL`	A subject can be unassigned from a year.
4	`users`	`subjects`	`subjects.profesor_id`	zero or one user → many subjects	`SET NULL`	Cached primary professor; assignment remains in `subject_professors`.
5	`users`	`subject_professors`	`subject_professors.profesor_id`	one user → many assignments	`CASCADE`	A professor can teach several subjects.
6	`subjects`	`subject_professors`	`subject_professors.subject_id`	one subject → many assignments	`CASCADE`	A subject can have several professors.
7	`users`	`materials`	`materials.autor_id`	one user → many materials	`CASCADE`	Every material has one author.
8	`materials`	`material_subjects`	`material_subjects.material_id`	one material → many associations	`CASCADE`	A material can be published in several subjects.
9	`subjects`	`material_subjects`	`material_subjects.subject_id`	one subject → many associations	`CASCADE`	A subject can contain many materials.
10	`users`	`subscriptions`	`subscriptions.alumno_id`	one student → many subscriptions	`CASCADE`	A student can follow several subjects.
11	`subjects`	`subscriptions`	`subscriptions.materia_id`	one subject → many subscriptions	`CASCADE`	A subject can be followed by many students.
12	`users`	`audit_logs`	`audit_logs.usuario_id`	zero or one user → many logs	`SET NULL`	Actor is optional and preserved as `NULL` if the user is hard-deleted.

#	Parent	Child	Foreign key	Cardinality	On delete	Meaning
13	`users`	`thematic_tags`	`thematic_tags.creado_por`	zero or one user → many tags	`SET NULL`	Tag creator is optional.
14	`users`	`material_favorites`	`material_favorites.usuario_id`	one user → many favorites	`CASCADE`	A user can favorite many materials.
15	`materials`	`material_favorites`	`material_favorites.material_id`	one material → many favorites	`CASCADE`	A material can be favorited by many users.
16	`users`	`notifications`	`notifications.usuario_id`	one user → many notifications	`CASCADE`	A user receives many notifications.
17	`subjects`	`notifications`	`notifications.subject_id`	one subject → many notifications	`CASCADE`	Notifications identify the subject that originated them.
18	`materials`	`notifications`	`notifications.material_id`	one material → many notifications	`CASCADE`	Notifications point to the new material.

4. Table Dictionary

> `yes*` marks `created_at` columns whose DDL omits `NOT NULL`. Their effective nullability depends on the server: MySQL 8 with `explicit_defaults_for_timestamp=ON` permits `NULL`, while MariaDB defaults those `TIMESTAMP` columns to `NOT NULL` unless the same variable is enabled. The application always relies on the `CURRENT_TIMESTAMP` default. `audit_logs.timestamp` is explicitly `NOT NULL`.

4.1 `roles`

Purpose: Role catalog used for authorization (`Superadmin`, `Profesor`, `Alumno`).

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Role identifier.
`nombre`	`VARCHAR(50)`	no	—	UNIQUE	Role name.
`descripcion`	`VARCHAR(255)`	yes	`NULL`	—	Human-readable role description.

Seeded roles: `Superadmin` (1), `Profesor` (2), `Alumno` (3).

Note: Roles are seeded by `init.sql`; there is no public role-creation endpoint.

4.2 `users`

Purpose: All application accounts: administrators, professors and students.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	User identifier.
`role_id`	`INT`	no	—	FK → `roles.id`, index	Assigned role.
`nombre`	`VARCHAR(50)`	no	—	—	First name.
`apellido`	`VARCHAR(50)`	no	—	—	Last name.
`email`	`VARCHAR(100)`	no	—	UNIQUE	Login address. Registration, login and reset normalize it; admin professor creation stores the submitted value as-is.
`password`	`VARCHAR(255)`	no	—	—	bcrypt hash.
`reset_token`	`VARCHAR(255)`	yes	`NULL`	—	Hashed password-reset token.
`token_expiration`	`DATETIME`	yes	`NULL`	—	Expiration for the reset token.
`verification_token`	`VARCHAR(255)`	yes	`NULL`	—	Hashed account-verification token.
`verification_expires`	`DATETIME`	yes	`NULL`	—	Expiration for the verification token.
`estado`	`ENUM('Activo','Inactivo','Baneado','PendienteVerificacion')`	no	`'Activo'`	—	Account lifecycle state.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Registration timestamp.

State meanings

- `Activo`: can log in and use the system.
- `Inactivo`: soft-deleted account; cannot log in.
- `Baneado`: administratively blocked; cannot log in.
- `PendienteVerificacion`: registered but not yet verified; cannot log in.

Seeded users

ID	Role	Email	Purpose
1	Superadmin	`admin@bibliotech.edu`	Initial administrator (`Sofia Administradora`).
2	Profesor	`profesor@bibliotech.edu`	Initial professor (`Pablo Profesor`).
3	Alumno	`alumno@bibliotech.edu`	Initial student (`Ana Alumna`).

All seeded passwords use bcrypt hashes. The seed comments document `Admin123`, `Profesor123` and `Alumno123`; these are development credentials and must be changed before any real deployment.

Indexes: unique `email`; index on `role_id`.

4.3 `careers`

Purpose: Academic programs (for example, “Ingeniería de Sistemas”).

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Career identifier.
`nombre`	`VARCHAR(100)`	no	—	UNIQUE	Career name.
`color`	`VARCHAR(7)`	no	`'#6f5bd3'`	—	UI accent color.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Creation timestamp.

Indexes: unique `nombre`.

4.4 `career_years`

Purpose: Ordered year/semester levels inside a career (for example, “Semestre 1”).

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Year-level identifier.
`career_id`	`INT`	no	—	FK → `careers.id`, `ON DELETE CASCADE`	Owning career.
`nombre`	`VARCHAR(100)`	no	—	UNIQUE with `career_id`	Level name.
`orden`	`INT`	no	`0`	—	Display order inside the career.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Creation timestamp.

Constraints: `UNIQUE (career_id, nombre)`.

Indexes: index on `career_id`.

4.5 `subjects`

Purpose: Subjects/materias and their placement in a career year.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Subject identifier.

Column	Type	Null	Default	Key / constraint	Description
`nombre`	`VARCHAR(100)`	no	—	—	Subject name.
`descripcion`	`VARCHAR(500)`	yes	`NULL`	—	Description.
`profesor_id`	`INT`	yes	`NULL`	FK → `users.id`, `ON DELETE SET NULL`, index	Cached primary professor.
`career_year_id`	`INT`	yes	`NULL`	FK → `career_years.id`, `ON DELETE SET NULL`, index	Academic year/level.
`estado_asociacion`	`ENUM('Activa','Pendiente')`	no	`'Activa'`	—	Whether the subject is fully active or awaiting association confirmation.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Creation timestamp.

Important: `profesor_id` is not the authoritative list. The full teaching assignment is stored in `subject_professors`. Application code calls `replaceProfessors()` and then synchronizes `profesor_id` to the first professor.

Seed data: 23 subjects distributed across three seeded career years. All seed subjects have `profesor_id = NULL` and no `subject_professors` rows until assignments are made.

Indexes: `profesor_id`, `career_year_id` (both created by their foreign keys).

4.6 `subject_professors`

Purpose: Many-to-many assignment of professors to subjects. This is the authoritative teaching relationship.

Column	Type	Null	Default	Key / constraint	Description
`subject_id`	`INT`	no	—	PK, FK → `subjects.id`, `ON DELETE CASCADE`	Subject.
`profesor_id`	`INT`	no	—	PK, FK → `users.id`, `ON DELETE CASCADE`, index	Professor.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Assignment timestamp.

Primary key: composite `(subject_id, profesor_id)`.

Indexes: `profesor_id` for “subjects taught by me” queries.

Intended invariant: `profesor_id` should reference a user with the `Profesor` role. Create/update/request flows check it; `confirm-association` can insert any existing user id because only the FK is enforced.

4.7 `materials`

Purpose: Learning resources uploaded or linked by users.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Material identifier.
`autor_id`	`INT`	no	—	FK → `users.id`, `ON DELETE CASCADE`, index	Author.
`titulo`	`VARCHAR(150)`	no	—	—	Display title.
`tipo`	`ENUM('PDF','EPUB','Word','Presentacion','Link_Drive','Youtube','Repositorio')`	no	—	—	Resource format.
`url_or_path`	`VARCHAR(500)`	yes	`NULL`	—	Stored file path or external URL.
`portada`	`VARCHAR(500)`	yes	`NULL`	—	Cover image path.
`file_hash`	`VARCHAR(128)`	yes	`NULL`	UNIQUE	SHA-256 of the uploaded file, or of the external URL for link materials; used for duplicate detection.
`categories`	`TEXT`	yes	`NULL`	—	Comma-separated category/tag text.
`descripcion`	`TEXT`	yes	`NULL`	—	Description.
`estado`	`ENUM('Aprobado','Pendiente','Cancelado','Rechazado')`	no	`'Pendiente'`	—	Moderation/publication state.
`es_favorito`	`TINYINT(1)`	no	`0`	—	Legacy/global favorite flag. Do not use for per-user state.
`contador_favoritos`	`INT`	no	`0`	—	Denormalized favorite count.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Upload timestamp.

State meanings

- `Aprobado`: visible in the global catalog; subject pages additionally require an approved association.
- `Pendiente`: uploaded by a student, awaiting professor review.
- `Cancelado`: withdrawn by the author; file fields are cleared and associations removed.

- `Rechazado`: a pending student upload rejected by a professor; file fields are cleared.

Type values

`PDF`, `EPUB`, `Word`, `Presentacion`, `Link_Drive`, `Youtube`, `Repositorio`.

Indexes: `autor_id`; unique `file_hash`; application queries also filter by `estado`, `tipo`, and title.

File storage: `url_or_path` and `portada` are typically relative paths under `uploads/`, but external links use the full URL. Storage mode is configured by `FILES_MODE` (`local` or `cloud`).

4.8 `material_subjects`

Purpose: Many-to-many classification of materials into subjects, including per-subject moderation state.

Column	Type	Null	Default	Key / constraint	Description
`material_id`	`INT`	no	—	PK, FK → `materials.id`, `ON DELETE CASCADE`	Material.
`subject_id`	`INT`	no	—	PK, FK → `subjects.id`, `ON DELETE CASCADE`	Subject.
`estado_aprobacion`	`ENUM('Aprobado','Pendiente')`	no	`'Pendiente'`	—	Moderation state for this material-subject pair.

Primary key: composite `(material_id, subject_id)`.

Behavior

- Professor uploads are inserted as `Aprobado`.
- Student uploads are inserted as `Pendiente`.
- Catalog queries require **both** `materials.estado = 'Aprobado'` and `material_subjects.estado_aprobacion = 'Aprobado'`.
- A material can be approved for one subject while still pending in another. A student duplicate of an already-approved material adds a `Pendiente` association without changing the material's global `Aprobado` state.

Indexes: `subject_id`; composite PK covers `material_id` lookups.

4.9 `subscriptions`

Purpose: Students following subjects. Subscribers are the observers notified about new material.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Subscription identifier.
`alumno_id`	`INT`	no	—	FK → `users.id`, `ON DELETE CASCADE`, index	Student.

Column	Type	Null	Default	Key / constraint	Description
`materia_id`	`INT`	no	—	FK → `subjects.id`, `ON DELETE CASCADE`, index	Subject.
`estado`	`ENUM('Suscrito','Baneado')`	no	`'Suscrito'`	—	Subscription state.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Subscription timestamp.

Constraints: `UNIQUE (alumno_id, materia_id)`.

State meanings

- `Suscrito`: receives notifications and appears in the observer list.
- `Baneado`: blocked by a professor; cannot re-subscribe while the row exists.

Indexes: unique `(alumno_id, materia_id)`; indexes on both foreign keys.

4.10 `audit_logs`

Purpose: Immutable business-operation trail. Used by the admin audit screens.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Log identifier.
`timestamp`	`DATETIME`	no	`CURRENT_TIMESTAMP`	—	Event time.
`tipo_operacion`	`VARCHAR(100)`	no	—	—	Operation label.
`usuario_id`	`INT`	yes	`NULL`	FK → `users.id`, `ON DELETE SET NULL`, index	Actor, when known.
`registro_id`	`VARCHAR(100)`	yes	`NULL`	—	Polymorphic target identifier.

Stored operation labels used by the application

`Alta Usuario`, `Verificacion Cuenta`, `Solicitud Reseteo Clave`, `Cambio Clave`, `Desbaneo Usuario`, `Baja Usuario`, `Alta Rol`, `Solicitud Materia`, `Alta Materia`, `Asociacion Materia`, `Modificacion Materia`, `Alta Carrera`, `Modificacion Carrera`, `Alta Anio`, `Modificacion Anio`, `Subida Material`, `Asociacion Material`, `Aprobacion Material`, `Modificacion Material`, `Desasociacion Material`, `Cancelacion Material`, `Borrado Material`, `Suscripcion Materia`, `Baneo Alumno`.

The code defines symbolic keys in `AuditLog.OPERACIONES` (for example `ALTA_MATERIA`), but the value written to `tipo_operacion` is the human-readable label (for example `Alta Materia`).

Note: Login is not audited. Registration, verification, password reset, moderation, uploads, subscriptions and administrative changes are audited.

Why `registro_id` is not a foreign key: the target table depends on the operation (user, subject, material, role, career or career year). The audit service enriches logs in application code by looking up the target according to the operation.

4.11 `thematic_tags`

Purpose: Moderated tag vocabulary for categorizing materials.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Tag identifier.
`nombre`	`VARCHAR(100)`	no	—	UNIQUE with `categoria`	Tag name.
`categoria`	`VARCHAR(100)`	no	—	UNIQUE with `nombre`	Tag group; the service supplies `general` when omitted.
`creado_por`	`INT`	yes	`NULL`	FK → `users.id`, `ON DELETE SET NULL`, index	Creator.
`baneado`	`TINYINT(1)`	no	`0`	—	Banned tags are listed but cannot be created again.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Creation timestamp.

Constraints: `UNIQUE (nombre, categoria)`.

No FK to materials: materials store tags in `categories` text. Creating a tag does not attach it to a material automatically; the UI/app sends the text values.

4.12 `material_favorites`

Purpose: Per-user favorite relationship.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Favorite identifier.
`usuario_id`	`INT`	no	—	FK → `users.id`, `ON DELETE CASCADE`, index	User.
`material_id`	`INT`	no	—	FK → `materials.id`, `ON DELETE CASCADE`, index	Material.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	—	Favorite timestamp.

Constraints: `UNIQUE (usuario_id, material_id)`.

Behavior: Toggling a favorite inserts/deletes this row and updates `materials.contador_favoritos`. Catalog responses expose `esFavorito` based on this table, not on `materials.es_favorito`.

4.13 `notifications`

Purpose: In-app notifications for students when a professor publishes or associates a material in a subject they follow.

Column	Type	Null	Default	Key / constraint	Description
`id`	`INT`	no	auto increment	PK	Notification identifier.
`usuario_id`	`INT`	no	—	FK → `users.id`, `ON DELETE CASCADE`, index	Recipient.
`subject_id`	`INT`	no	—	FK → `subjects.id`, `ON DELETE CASCADE`	Subject that originated the notification.
`material_id`	`INT`	no	—	FK → `materials.id`, `ON DELETE CASCADE`	New material.
`created_at`	`TIMESTAMP`	yes*	`CURRENT_TIMESTAMP`	index	Creation time.
`read_at`	`DATETIME`	yes	`NULL`	—	Read timestamp; `NULL` means unread.

Constraints: `UNIQUE (usuario_id, subject_id, material_id)`.

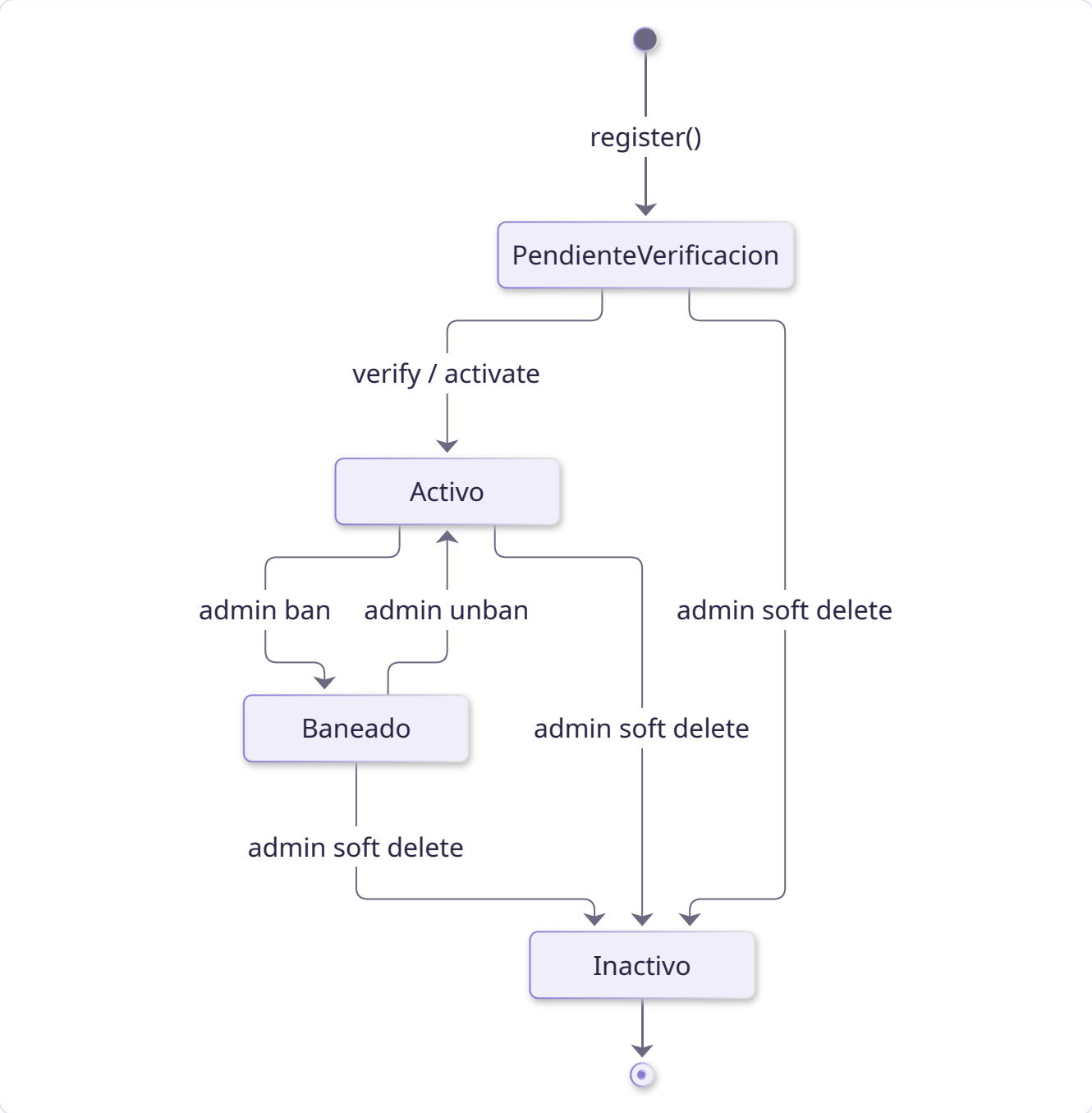
Behavior

- One row per subscriber per material per subject.
- If the same material is published again to the same subject, the existing row is refreshed (timestamp updated, `read_at` cleared) instead of duplicated.
- `GET /api/notifications` returns the latest 30 notifications and an `unreadCount`.
- Notifications are created only for students with `subscriptions.estado = 'Suscrito'`. The notification path does not re-check `materials.estado`, so a professor can notify about a material that is still `Pendiente` (for example, by associating an existing pending material).

Indexes: `(usuario_id, read_at, created_at)` for inbox queries; indexes on all three foreign keys.

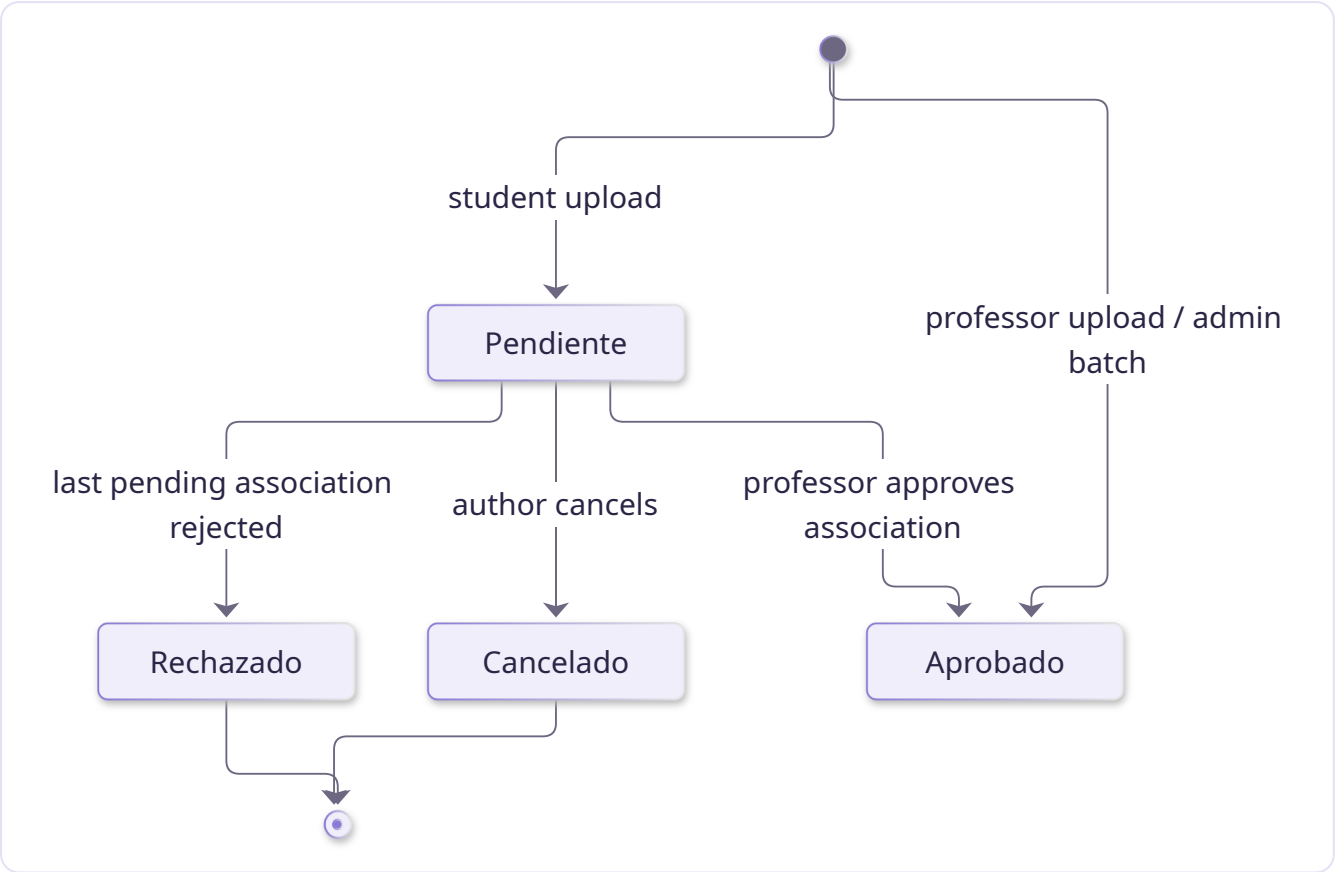
5. State Machines and Lifecycles

5.1 Account lifecycle



The admin ban/unban endpoints do not restrict the starting state: `ban` can set any existing account to `Baneado`, and `unban` can set any existing account (including `Inactivo`) back to `Activo`.

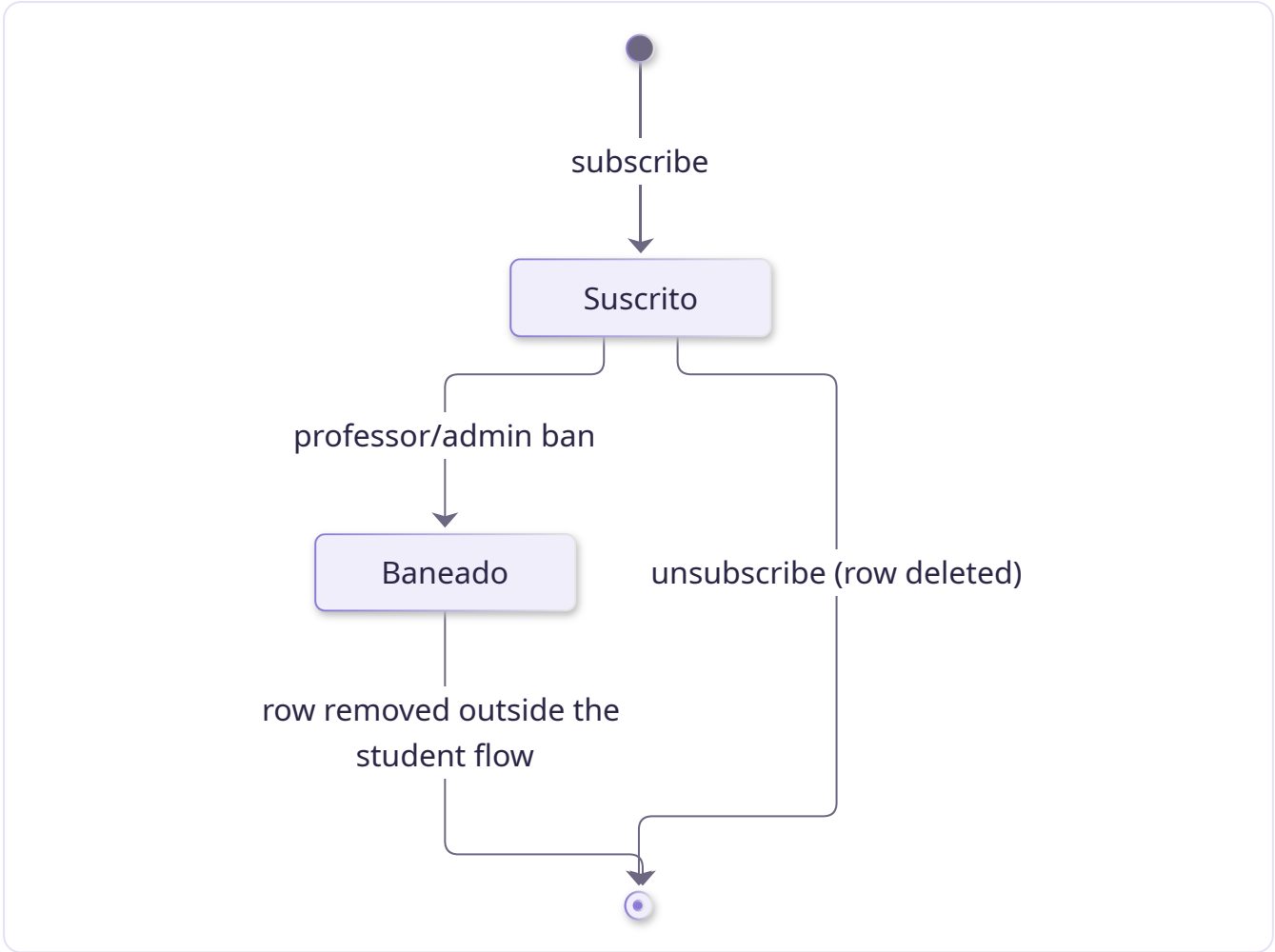
5.2 Material and association lifecycle



For a material with several subjects, `materials.estado` and each `material_subjects.estado_aprobacion` can differ:

- Student upload: material `Pendiente`; every association `Pendiente`.
- Professor approves one subject: that association becomes `Aprobado`; the material becomes `Aprobado`.
- Professor rejects one subject: that association is deleted. If other associations remain, the material stays visible for those subjects. If no associations remain and the material was `Pendiente`, the material becomes `Rechazado` and its file fields are cleared. If the material was already `Aprobado`, it remains `Aprobado` with no associations and its files are kept.
- Cancel: only `Pendiente` materials can be canceled by the author. The material becomes `Cancelado`, file fields are cleared, and all associations are deleted.

5.3 Subscription lifecycle



The service refuses to subscribe a student whose row is `Baneado`; the ban persists until the row is removed or changed outside the normal student flow.

6. Derived Counts and Catalog Semantics

The repositories compute these values rather than storing them:

Value	Meaning
`career.totalMaterias`	Number of subjects in the career.
`career.totalMateriales`	Number of distinct material ids in approved associations within the career; the query does not additionally filter `materials.estado`.
`career_year.totalMaterias`	Number of subjects in that year.
`career_year.totalMateriales`	Number of distinct material ids in approved associations for that year; no additional `materials.estado` filter.
`subject.totalMateriales`	Number of associations with `estado_aprobacion = 'Aprobado'`; no additional `materials.estado` filter.

Value	Meaning
`material.contadorFavoritos`	Stored counter, updated on favorite toggle.
`material.esFavorito`	Whether the current authenticated user has a `material_favorites` row.

Catalog visibility rules:

1. **Global catalog:** `materials.estado = 'Aprobado'`. An approved material remains visible globally even if it has no subject associations, so professors can find and re-associate it.
2. **Subject page:** `materials.estado = 'Aprobado'` **and** a matching `material_subjects` row with `estado_aprobacion = 'Aprobado'`.
3. **Author profile:** approved materials authored by that user.

7. Migrations and Compatibility

Script	What it changes
`init.sql`	Drops and recreates the complete schema, seeds roles, users, one career, three year levels and 23 subjects. Intended for a fresh database.
`migrate_academic_groups.sql`	Adds `careers` and `career_years`; adds `subjects.career_year_id`; adds new material types, `materials.portada`, favorites and notifications; maps known subjects into career years.
`migrate_refinements.sql`	Adds `subject_professors`; backfills it from `subjects.profesor_id`; makes material URLs/hashes nullable; adds the `Rechazado` state.
`migrations/001_notifications.sql`	Creates the `notifications` table for existing installations.

The current schema is the union of all four. On a fresh install, `init.sql` already contains the final shape; the migration scripts are for upgrading an older database.

8. Integrity Rules and Application Responsibilities

Rule	Enforced by
Email uniqueness	Database unique index + `AuthService`.
Role existence	FK `users.role_id`.
Only users with role `Profesor` should be assigned to subjects	Application checks on subject create/update and professor request; `confirm-association` with an arbitrary `profesorId` only relies on the user FK and does not re-check the role.
A professor can only review materials in subjects they teach	Application (`Subject.hasProfessor`).

Rule	Enforced by
A student can only upload three pending materials	Application (`MaterialService.uploadOrSuggestMaterial` / `assertPendingLimit`).
A student cannot subscribe to a banned subscription	Application (`SubscriptionService.subscribe`).
A professor can only ban observers of subjects they teach	Application (`SubscriptionService.banStudent`).
Duplicate active files are not stored twice	Database unique `file_hash` + application duplicate flow.
One favorite per user/material	Database unique constraint.
One notification per user/material/subject	Database unique constraint.
Tags are unique per `(nombre, categoria)`	Database unique constraint.
Audit rows are append-only	Application; no update/delete endpoint exists.
Soft-deleting a user preserves history	Application sets `estado = 'Inactivo'`; hard deletion cascades.

Application-only invariants

- `subjects.profesor_id` should equal one of the professors in `subject_professors`; the repository synchronizes it after assignment changes.
- `materials.contador_favoritos` should equal the number of rows in `material_favorites` for that material; the favorite flow increments/decrements it without an explicit transaction, and the API returns a fresh count.
- `materials.es_favorito` is not kept in sync with per-user favorites and should be treated as legacy.
- A material with no `material_subjects` rows is not shown on subject pages, but it remains visible in the global catalog when its `estado` is `Aprobado`.

9. Deletion Behavior Summary

Deletion	Effect
Delete a role	Blocked while users reference it (no cascade).
Delete a career	Deletes its `career_years`; subjects in those years get `career_year_id = NULL`.
Delete a career year	Subjects get `career_year_id = NULL`.
Delete a subject	Deletes its `subject_professors`, `material_subjects`, `subscriptions`, and `notifications`; materials remain.

Deletion	Effect
Delete a user (hard)	Deletes authored materials, subscriptions, favorites and notifications; audit logs and tags keep `NULL` actor/creator; subject assignments and cached `profesor_id` are cleared.
Delete a material	Deletes its `material_subjects`, `material_favorites`, and `notifications`; files are removed by the application.
Delete a subscription	Notifications already created remain.
Delete a tag	Materials keep their text categories.

The public API avoids hard-deleting users: `DELETE /api/auth/users/:id` sets `estado = 'Inactivo'`.

10. Seed Data Summary

`init.sql` seeds:

- 3 roles.
- 3 users (admin, professor, student).
- 1 career with 3 year levels.
- 23 subjects:
  - Year 1: 8 subjects.
  - Year 2: 8 subjects.
  - Year 3: 7 subjects.
- No `subject_professors` assignments; all subjects start with `profesor_id = NULL`.
- No materials, subscriptions, favorites, tags, notifications or audit rows.

This means a fresh database has a populated academic catalog but no teaching assignments. An administrator must create/assign professors before professor-specific flows can return meaningful data.